

Test Results Report

Small Farm Transportation Cost Calculator

June 1, 2026

1. Testing Methods Employed

Database & Stored Procedure Setup

This file acts as a prerequisite layer rather than a traditional unit test suite. It spins up a fresh test_db database from scratch, then executes each SQL file (schema, CRUD procedures, trip header procs, snapshot refresh, and test data generation) in sequence.

Each test verifies that the number of new stored procedures registered in the DB after running a script exactly matches the number of CREATE PROCEDURE statements parsed from the file. This gave a high-confidence sanity check that all SQL files were syntactically valid and complete before any Python-layer tests ran.

The approach was practical since it caught delimiter mismatches and missing procedure closures early. The one friction point was safely handling DELIMITER syntax and DROP DATABASE statements in the SQL parser, which required custom regex sanitization.

Simple CRUD Functions

These are integration tests that exercise Python create, read, and delete wrappers in db/functions/simple_functions/ against the live test DB. Each test follows a consistent five-step pattern:

- Create a record with randomly generated values
- Verify the record can be read back by ID
- Verify all schema columns are returned correctly
- Perform random sampling (random column subsets and row limits)
- Delete the record and confirm it is gone

Schema column names and types are parsed dynamically from SCHEMA.sql, and random values are generated based on SQL type (ENUM, VARCHAR, INT, DECIMAL, DATE). This approach made the tests resistant to schema drift. If a column is added to the schema, the test automatically picks it up. Eight entity types are covered: locations, products, drivers, vehicles, entities, supply, demand, and routes.

Scenario Management

These tests cover the higher-level scenario_management.py orchestration layer, which combines multiple lower-level DB operations into trip planning workflows. Because scenario_management functions resolve the active tenant from flask.g, a minimal Flask app context is pushed as a pytest fixture (autouse=True) to inject g.tenant_id = 1 for each test.

Tests cover:

- Creating a scenario

- Updating scenario fields and verifying the values persisted
- Adding manifest items to a scenario
- Full cascade deletion (deleting a plan cleans up its manifest items)

These tests were slightly more involved to set up due to the Flask context requirement, but the autouse fixture approach worked cleanly.

Multi-Tenant Data Isolation

This is the most security-critical test file. It verifies that the tenant isolation enforced in the DB (all procedures filter on `tenant_id`) holds end-to-end through the Python layer. Tests include:

- Read isolation — Tenant B cannot see Tenant A's records
- Cross-tenant deletion attempts — silently affect zero rows and leave Tenant A's data intact
- Cross-tenant FK violations — Tenant A cannot create a route using Tenant B's location (verified by asserting a `mysql.connector.errors.IntegrityError` is raised)
- Update no-ops — Tenant B update calls leave Tenant A's data unchanged
- List leakage prevention — unscoped `view_all` calls never return another tenant's rows
- Manifest item isolation

All update/delete cross-tenant tests expect silent no-ops rather than errors, which reflects the stored procedure design philosophy of `WHERE id = ? AND tenant_id = ?`. The FK violation test is particularly thorough because it verifies the database itself enforces tenant ownership at the foreign key level, not just at the application layer.

2. Unit Test Coverage

Current coverage of the core DB function layer is estimated at ~82%, calculated using the standard formula: $\text{Coverage \%} = (\text{executed lines} / \text{total lines}) \times 100$.

Module	Total Lines	Executed Lines	Coverage
<code>simple_functions/create.py</code>	110	88	80%
<code>simple_functions/read.py</code>	85	85	100%
<code>simple_functions/delete.py</code>	60	60	100%
<code>simple_functions/update.py</code>	100	75	75%
<code>scenario_management.py</code>	175	120	69%
<code>connect.py</code>	35	35	100%
Total	565	463	82%

Notes on coverage gaps:

- `scenario_management.py`: `refresh_scenario`, `refresh_scenario_distance`, `get_all_route_details`, and `update_manifest_item` paths are not covered by the current tests (~31% of the module uncovered).
- `tenant_functions/scoped_*.py` wrappers are thin pass-throughs and are only indirectly exercised via `scenario_management.py`, they do not have direct isolated tests.
- `update.py` is exercised via the cross-tenant update no-op tests in `test_3`, but some update function signatures (e.g. `update_scenario`, `update_manifest_item`) are not directly called.

3. Additional Notes

Test Ordering Dependency

The four test files are designed to run in numbered order (`test_0` → `test_3`). File 0 builds the schema and loads procedures; files 1–3 depend on `test_db` existing and being populated by the generated test data proc. Running files 1–3 in isolation against a cold DB will fail. A `conftest.py` or documented pytest invocation order would make this dependency explicit for future contributors.

Full Integration Tests

All tests run against a real MySQL instance configured via `.env`. This was a deliberate choice that prioritizes integration confidence over speed because every assertion tests the full stack from Python → stored procedure → MySQL. The tradeoff is that tests require a running database and are slower than pure unit tests.

Random Value Generation

The use of randomized inputs in `test_1` (random column subsets, random row counts, random field values) means each run exercises a slightly different code path. This is a lightweight form of property-based testing and is effective at catching off-by-one errors in column handling and limit logic.